

BridgeFlow AI

Multi-agent monthly business review on the official DeepSeek Harness

TEAM CODE	CA3WTLPW
EVENT	Show Me Your Agents Hackathon · NUS-ISS · September 2026
LIVE DEMO	https://47.130.178.176.sslip.io
SIGN-IN PORTAL	https://portal.47.130.178.176.sslip.io
REPOSITORY	github.com/EricWang1358/BridgeFlow-AI (private; access on request)
COMPANION	Business Proposal (problem, market, business model)

Every measured figure in this document has a reproduction command in `docs/00-status.md`; raw evidence artefacts live in `docs/evidence/`.

1 System overview and design principles

BridgeFlow reads the monthly spreadsheets that each department keeps in its own way, and turns them into an immutable batch plus a business review that shows its evidence. The reference scenario is a light-manufacturing SME with four departments (Production, Procurement, Finance, Marketing); fields, formulas, thresholds and each department's responsibilities are declarations in the field dictionary, not code, so the same engine carries other industries. This release fixes the department set to those four roles; a user-defined department set is the planned extension. It runs on the official DeepSeek Harness (`dsh`): native Web UI, sessions, approvals and concurrent per-department subagents. Python does the arithmetic the field dictionary declares; each department agent proposes actions inside its stated responsibility, and the host checks the structured findings before anything lands in a report.

Seven engineering constraints shaped the implementation; each is enforced in code review and, where possible, by a test:

1. **dsh is the base, not a provider.** The agent runtime's orchestration, sessions, approvals and subagents are used as-is; an earlier mistake — driving `dsh` as a completion backend — was measured (one adjudication: 12–212 s and $\approx 7,100$ tokens of tool output vs. 3.6 s and 707 tokens after correction), diagnosed, and reverted.
2. **No fork of dsh.** Product behaviour is customized through Client plugins and a pinned Web policy patch; the exact pinned CLI (`@deepseek-ai/dsh@0.1.2-rc.1`) is installed privately beside the repository so it can never collide with a user's own `dsh`.
3. **Raw data rows never enter model context** — enforced in three layers (tool output caps, projection-only summaries, request-side guards).
4. **No business field name is written into code;** a test parses the source to keep it true, so onboarding another company changes declarations, not code.
5. **Every conclusion carries evidence;** a finding without citations is refused at the schema layer.
6. **Bootstrap environment variables never come from files** — `DSH_*` and `DEEPSEEK_BASE_URL` only from the launching shell; this is a security boundary against hostile-repository privilege redirection.
7. **Reuse over rebuild:** the UI is the official `dsh` Web customized through plugins, not a self-built frontend; one CI workflow; no infrastructure beyond what the product needs.

2 Architecture

Official dsh Web: native chat, sessions, approvals, trajectory

- └ BridgeFlow slots: import/data, rejection note, review cards
 - └ Typed domain tools + host policy
 - └ Python: immutable batches, dictionary, arithmetic, validation
 - └ Official dsh spawn: production / procurement / finance / marketing
 - └ Native approval → one-use receipt → mapping memory

Component	Responsibility
scripts/start_web.py	Launcher: starts the private Python service (localhost:8000) and the official dsh web, issues the one-time browser credential, isolates DSH_HOME, and (guest mode) scrubs every operator secret out of the child environment before execve re-exec
backend/ (Python 3.12, FastAPI, pandas)	Domain computation and persistence: import, sanitizer, immutable batches, field dictionary, master table, metric arithmetic, validation, workflow and discovery domains, decision journal, LLM gateway
plugins/ (TypeScript)	dsh tools with typed schemas, request/approval guards, and Client UI slots (Sources, Studio, Business state, tours, user guide) — the UI is dsh Web customized, not a self-built app
dsh/	Pinned Web policy patch (enterprise preset) and a restricted analyst preset; shell access removed for subagents
portal/ (FastAPI)	Sign-in portal: Feishu OAuth, JWT issuance with JWKS, seat allocation, guest handover
deploy/	systemd units, Caddy configuration rendering, deploy and preflight scripts (35 checks)
.github/workflows/deploy.yml	The only CI file: offline checks on every PR; deploy to Lightsail plus health check on merge to main

Frontend surfaces. Three columns — Sources (uploaded files, previews with original row numbers and SHA-256 provenance), the native dsh chat (conversation, approvals), and Studio (domain pages: this month’s tasks, data, conclusions, records, filling & handoff, discovery, quotation, overview). Notebooks scope a session to its own sources; saving is explicit.

3 The monthly review loop

3.1 Import and the immutable batch

One file per department per month, CSV or XLSX, on the business’s own v2 templates. Multi-sheet workbooks and off-row headers are refused with a diagnosis, not guessed. Limits are enforced on total bytes, decompressed size, department count and row count (zip-bomb guard). The batch that results is **frozen**: a new import never recomputes an old batch, which is why citations still point where they pointed at import time. Replacing one department’s file derives a **new** batch version; the old one stays byte-identical as evidence.

3.2 Sanitizer

Rules first (type inference, date parsing, duplicate and shifted-header detection); the model handles only the residue. Every repair is logged with original value, new value, rule and confidence. Unfixable rows

are **quarantined** — kept verbatim, shown in the UI, and a batch with quarantined rows refuses to report totals, because a sum over a silently smaller table turns “at least this much” into “exactly this much”.

3.3 The field dictionary: rules as declarations

The dictionary (`field-dictionary.yaml`, per-customer master data, gitignored) declares which column of which department holds which entity, what may be computed, how source rows roll up, join columns, metric formulas, attention thresholds and dispositions. Three properties matter:

- **Frozen per batch.** Each batch stores an `integration_snapshot`; changing policy later never rewrites an old batch (tested).
- **Human-approved lifecycle.** The model can transcribe the business’s OA dictionary spreadsheet (`dictionary_import`) or draft from column profiles (`dictionary_draft`); every entry — including its rollup — is decided in an approval; `dictionary_publish` versions it for later imports.
- **Conventions are visible.** Where the dictionary is silent and a common practice was assumed (e.g. VAT 13%), the assumption is declared in `integration.yaml`, marked on every dependent cell, and listed for the business to confirm or replace — confirming upgrades evidence grades without changing any number.

3.4 Master table and open items

Rows meet on declared join columns (project code, customer code, report month). Declared formulas are computed and compared with what departments wrote. Disagreements leave the cell empty and become a question listing what each department wrote — the system never picks a spelling. Every convention, disagreement, quarantine and missing template column flows into one open-items inbox with “how to settle” guidance.

3.5 The cross-department review (multi-agent orchestration)

1. `review_context` reads the frozen batch packet and issues four one-use dispatch tickets.
2. The captain model calls the official `subagent` tool **four times in one response** — production, procurement, finance, marketing. The four child sessions have distinct session ids and genuinely overlap in time (visible in the trajectory view).
3. Each child may only submit `structured_output`. It has no shell, no editor, no code execution, no channel to the other three, cannot read the parent conversation, and cannot read the raw workbook.
4. `review_finalize` collects what the host actually recorded, so the parent model cannot upload four judgements of its own and pass them off as children’s results.
5. Python re-checks every value, unit, status, action and citation against the frozen dictionary. Anything that fails leaves that department **unvalidated** and the report honestly `partial`. A department that keeps producing invalid structure is stopped by a 3-step budget; the report then says that department has no valid judgement, keeps the other three, and manufactures no substitute.

Measured (real model, 2026-09-25, deepseek-v4.1-flash): risk case 4/4 departments validated, 53.4 s, 8 model requests, 75,267 tokens (44,416 cache reads). Balanced case 18.8 s / 106,485 tokens; injection case 12.3 s / 71,686 tokens (2026-09-23 runs). A partial-report path is exercised in CI with `BRIDGEFLOW_TEST_FAULT=step-limit`.

3.6 Report, evidence grades and dispositions

Each department card shows the responsibility, decision owner, metric with its value, the suggested action (copied verbatim from the declared check's `attention` branch — never model prose), the formula exactly as declared, and the attention threshold. The one model-authored sentence is marked `model_advice`; the execution status is always `proposed_only`. Citations carry file, original row and original column; original row numbers survive blank-row removal, de-duplication and quarantine. Conclusions carry evidence grades (G1 direct source, G2 declared-and-confirmed, G3 declared-unconfirmed, G4 model advice), and the report header states its own scope and limitations. Dispositions follow a dictionary-declared state machine; no default lifecycle is invented where the business has not decided one. The brief exports to Word (.docx) generated from the conclusion page, so the document cannot say more than the screen.

4 Human-in-the-loop machinery

Native approvals everywhere. Every write tool — mapping confirmation, disposition action, workflow submit/handoff/complete, dictionary publish, discovery saves — blocks on dsh's native approval panel with its arguments visible. Three endings, each semantically distinct and tested:

- **Allow once:** the write happens through a one-use receipt generated after the decision; replays are refused.
- **Reject with a reason:** nothing is written; the reason returns to the model, which must relay it. The tool-result wording was hardened after a live run caught the model claiming “will not be asked next month” — false; the wording now says future imports may ask again, and a live test pins the sentence.
- **Timeout:** writes nothing and is reported as `cancelled` — never as “a person refused”. `cancelled`, `unavailable` and `rejected` are three different facts; inventing a decision-maker is worse than the bug it papers over.

One distinction worth noting: `accepted=false` as a **tool argument** means “record the decision that this relation does not hold” (still a write, still needs approval); the **Reject button** means “do not run this call at all”.

Escalation design. Deterministic transforms run automatically and are logged; suggested repairs carry confidence and reason; critical fields are never auto-filled; low-confidence mappings queue for a person; quotes compute but never send. A human review note reaches the captain as an ordinary chat message and is plugin-guarded: a turn carrying a note cannot start, rerun or finalize a review, dispatch departments or call an approval tool. Every review registers a deadline when it opens; a late success cannot overwrite `deadline_exceeded`; a restart closes open runs as `host_restarted`.

5 The three-agent workflow (discovery → execution)

The business side designed three agents. Agent 1 (discovery): materials are registered behind approval with checksums and quotas; candidate work scenarios are drafted; an information/file-flow diagram records confirmed, inferred and missing links explicitly; four-quadrant scoring follows a human-declared policy (incomplete scores do not land; policy or source changes mark scores stale); meeting minutes version under native approval; the MVP decision is recorded with per-person votes — thresholds do not auto-approve, and conditions do not auto-release. Agent 2 (filling & handoff): the **approved MVP decision becomes the workflow's scope** — accepted through the captain behind approval, only at the version the person saw, only for scenarios the catalogue can run; a revised or withdrawn decision marks the scope

stale and new records are refused. Records then flow department to department over two demonstrated hops (production → marketing → finance): each handoff carries a due time from its stage and shows when overdue; a stage cannot complete until its own output is recorded under the same business key (a rule added because the real model **refused** to complete early — the offline scripted model had happily done so); every record keeps a timeline of who did what and when. Agent 3 (adoption) exists today as role guidance generated from declared templates; pilot pages and the feedback loop are declared but not built.

Measured (real model, 2026-09-24): the whole workflow — scope acceptance plus both hops — 8 native approvals, 28 model requests, 76.9 s; per-step timings from 4.3 s to 11.8 s; token totals dominated by cache hits (445,440 of ≈484 k).

6 Safety and security

6.1 Data isolation (three layers)

Raw data rows never enter model context. (1) Tool outputs are capped and contain counts, formulas and bounded evidence samples — not table dumps; a test pins that numeric conflict values never reach the model summary. (2) The browser pages through data; the model has no path to raw rows. (3) At 200 k rows, tool outputs stay ≤ 37 KB (measured). The decision journal records **no** data rows (verified by per-value comparison against master-table text), and the LLM gateway logs counts only.

6.2 Prompt-injection defence (measured, not asserted)

Spreadsheet cell content is data, never instruction — four files maintained by four different people is exactly the trust problem cross-department data has. Bilingual injection cases are planted in a demo variant (BRIDGEFLOW_POISON=1); on the real model, the injected text reached **no** model session while all four departments still validated (12.3 s, 71,686 tokens). The attacker’s goal — tool calls from injected instructions — is additionally blunted because every write needs an approval a human sees.

6.3 Identity, roles and audit

Sign-in is per-employee Feishu OAuth through the portal; the shared service token was retired as an identity and is transport-only. The backend verifies portal JWTs via JWKS on every model-initiated read; the audit subject is user:<digest>, host OR anonymous — two simultaneously-bound users attribute to **nothing** rather than to a wrong name. Roles resolve from Feishu wiki membership through one access-control.yaml (repository-tracked after a deploy-channel incident was root-caused); the agent console sits behind an operator role gate with fail-closed 503 semantics and a 60-second cache that never caches failures. Model-initiated writes require role permission **and** a native approval.

6.4 The public demo: five-layer key defence

The zero-login evaluator entry runs a separate guest stack — its own backend (8001) and console (3090), data/guest/ wiped at startup, a 03:30 nightly reset, six upload entry classes returning 403, Feishu credentials always stripped, AI off by default. When AI is on:

Layer	Control
L0	Dedicated prepaid model account with no auto top-up — a physical spend ceiling independent of our code

Layer	Control
L1	The real key exists only in the <code>llm-gate</code> process (localhost:8300). Guest processes receive only the gate's address and a worthless client token. The launcher strips every operator secret and <code>execve</code> re-execs itself with the scrubbed environment, so even <code>/proc/<pid>/environ</code> of the launcher is clean — a hole the online preflight caught and the local run had missed
L2	Gate policy (tracked YAML): model whitelist, <code>max_tokens</code> cap (32,768), request-size cap (2 MB), 60 rpm, 6 concurrent, optional daily token budget, kill-switch file
L3	Application: review cooldowns, captain per-turn tool limits, model picker disabled for guests
L4	Per-request accounting in SQLite (counts only, no bodies); preflight asserts the gate listens on loopback only and guest env contains no operator key

7 Observability and evaluation

Decision journal. One middleware seam records every backend decision: method, path, subject, duration, result — rejections verbatim in their own words. Measured: 11 requests → 1 write, 6 reads, 4 refusals; median 45.9 ms; ≈271 bytes per entry; 14-day retention as daily JSONL. A Records page shows refusal wording rankings, filtering by result and batch, each entry carrying the same trace id as the response header.

Agent-run tracing. Every model request's call tree is journalled (`x-bridgeflow-root`); the run view renders per-agent swimlanes — markers are tool calls, width is duration share, refusals are dashed with `?` (never colour-only). Browser-initiated reads are excluded by construction.

Acceptance evaluation. `python -m bridgeflow.eval` runs 24 golden-path cases across three industries plus 7 adversarial/refusal cases (planted instructions, no base period, plan without source, undeclared steps, wrong-month files, single-period trend, source-less convention): **31/31 passing**; the report carries generation time and age, red rows name their owning issue.

Tool-selection evaluation (agent quality, not code quality). Real-model cases ask the captain an ordinary business question and check the **first tool** it picks among ≈50. Current: 21/22 (expanded to 22 cases 2026-09-25), median 3 steps and ≈41 k tokens per case. The three earlier misses traced to tool contracts and persona wording, not the model, and were fixed there — with the scoring standard fixed in advance and never edited after the fact.

Honest-status discipline. `docs/00-status.md` is the single source for every measured number (instituted after two contradictory copies of the same test count coexisted); each run adds its reproduction command, and the README's status table separates offline-scripted evidence from real-model evidence.

8 Testing and delivery pipeline

Suite	Scope
Backend pytest — 779 passing	Rules, contracts, demo-case expectations, workflow state machine, boundary regressions (frozen snapshots, header offsets, non-finite cells, formula-shaped text kept as text), security (injection, approvals, identity), no-field-names-in-code guard

Suite	Scope
Portal pytest — 47 passing	OAuth, JWT/JWKS, seat allocation, guest handover, console role gate
Plugin suite — 115 tests	Tool catalogue, copy keys, approval display completeness, guards; TypeScript typecheck and build
Browser journeys (offline, scripted model)	tour-smoke (12-step in-page tour), round1-journey, web-smoke (discovery chain), business-smoke (review + fault injection), workflow-journey, quotation-smoke, cases-journey, cold-reload, notebook walkthrough, guide journey — zero page errors required
Fault injection	BRIDGEFLOW_TEST_FAULT=step-limit forces an invalid department to prove the partial-report path; BRIDGEFLOW_POISON=1 plants injection cells
CI/CD	The single workflow file runs offline checks per PR; on merge to <code>main</code> it deploys to Lightsail and health-checks; a 35-check preflight (TLS, services, Caddy, gate loopback-only, clean environment, working tree) gates the instance

The offline scripted model makes every journey free and deterministic — and the team’s standing caveat is written down: scripted passes prove the code holds, not model quality; the latter is what the real-model evidence in `docs/evidence/` is for.

9 Performance and scale (measured)

Probe	Result
Four-department review (risk)	53.4 s wall clock, 8 requests, 75,267 tokens, 4/4 validated (2026-09-25)
Workflow, two hops	76.9 s, 8 approvals, 28 requests; cache hits 445,440 tokens (2026-09-24)
Captain tool selection	Median 3 steps / 41,407 tokens per question; 21/22 correct first tool (2026-09-25)
200,000-row CSV	34.1 MiB (over the 25 MiB upload cap; XLSX variant expands to 176 MiB and hits the zip-bomb guard). With limits raised: imports in 80.9–85.8 s, 199,988 rows kept, peak RSS 1,164 MiB; tool outputs $\leq$ 37 KB; summaries recompute in 7–9 s per call — limits and memory budget are an explicit operator decision
Per-seat footprint	Idle RSS $\approx$ 289 MiB through the full launcher path; cold start $\approx$ 1 s; 7 seats fit a 2 GB budget with headroom monitored by preflight
Decision journal overhead	Median 45.9 ms per request including the sample batch import at 2.1 s

10 Deployment and operations

AWS Lightsail (Ubuntu), Caddy for TLS and routing. Employees sign in at `portal.<domain>` (Feishu OAuth); each employee is allocated a seat — a per-user `dsh` process with its own `DSH_HOME` (FCFS pool of seven, quota'd) — reached through `forward_auth` with a WebSocket-upgrade fix that was root-caused end-to-end (a bare `GET /verify` returned 200 while an upgrade handshake returned 403 before the fix; the preflight now asserts the discriminating request). The main domain serves the public guest demo once the guest units are enabled; `/__enter` performs a token handover so evaluators never see a login box, with missing-token 503 retry semantics to avoid redirect loops. Nightly reset via `systemd` timer. Rollback: disable guest units and re-render Caddy; the production (Feishu) path is untouched by demo operations. Secrets live only in an untracked, operator-exported environment file; the preflight refuses to run without an explicit domain.

One production incident shows the pipeline learning from failure: an access-control file failed to deploy through the secret channel, the step's `set -e` semantics masked the failure, and health checks missed it (401-accepting probes). The fix made the file repository-tracked, moved the secret channel after deploy, added `set -eo pipefail`, and added a guard test.

11 Limitations and next steps

Not built, and not shown as if it were: per-employee isolation of native chat sessions; personal audit of rejections; formal report sign-off; risk tiers beyond read/approval; a real notification channel for handoffs; template generation from decisions; Feishu end-to-end import (metadata and membership reads verified on the real tenant); a third-party penetration test; multi-customer tenancy; a guided editor for fully user-defined departments; connectors beyond Feishu (Lark international sign-in, Google Workspace Sheets/Drive and other platforms), which are the next extension points of the same connector layer. All demo data is fictional and labelled as such; real customer exports, business confirmation of assumed conventions and enterprise acceptance are outstanding. The guest entry's server-side enablement and one billed gate-routed review were pending at submission time; everything else claimed above carries a reproduction command or an evidence artefact in the repository.

Repository references: `README.md` (guided evaluator walkthrough), `docs/00-status.md` (every measured number and its command).

Evidence artefacts: `docs/evidence/live-2026-09-2{3,4,5}/`.